

Spatial Data Science

Graduate – Elective (MSDS/MSCS/PhDCS)



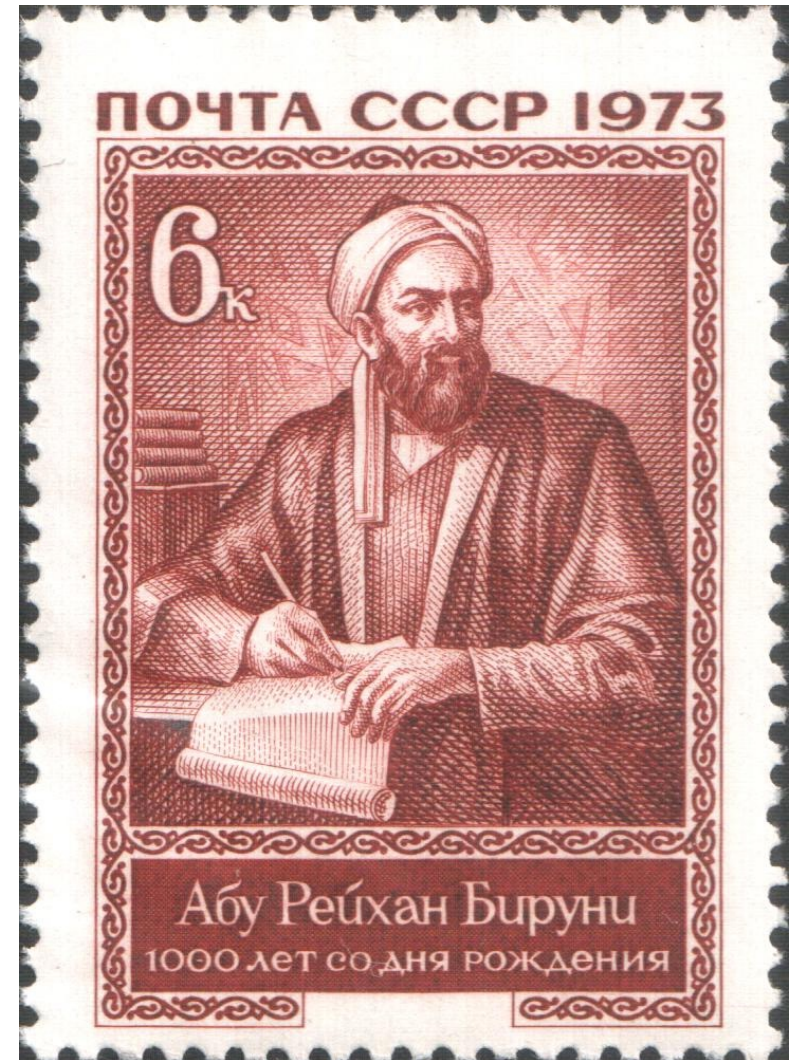
Remote Sensing
&
Spatial Analytics

Dr. sc. Muhammad Adnan Siddique
Assistant Professor
Information Technology University - ITU

Father of modern geodesy

Abu Rayhan Muhammad ibn Ahmad al-Biruni

(CE 973 –1050)



By The original uploader was Romanm at Slovenian Wikipedia. - Originally from sl.wikipedia; description page is/was here., Public Domain, <https://commons.wikimedia.org/w/index.php?curid=2590841>

Spatial Weights

“Spatial weights” are one way to represent graphs in geographic data science and spatial statistics.

Spatial weight matrices

- Spatial weights often express our knowledge about spatial relationships.
- What neighborhoods are you surrounded by? How many gas stations are within 5 miles of my stalled car?
- For us to use this information in statistical analysis, it's often necessary to compute these relationships between all pairs of observations.
- This means that, for many applications in geographic data science, we are building a topology.

Spatial weights matrices



Pysal

- Spatial weights are widely used throughout spatial and geographic data science.
- We illustrate all of these concepts through the spatial weights class in `pysal`, which provides a rich set of methods and characteristics for spatial weights and it is stored under the `weights` sub-module:

```
from pysal.lib import weights
```



Contiguity weights

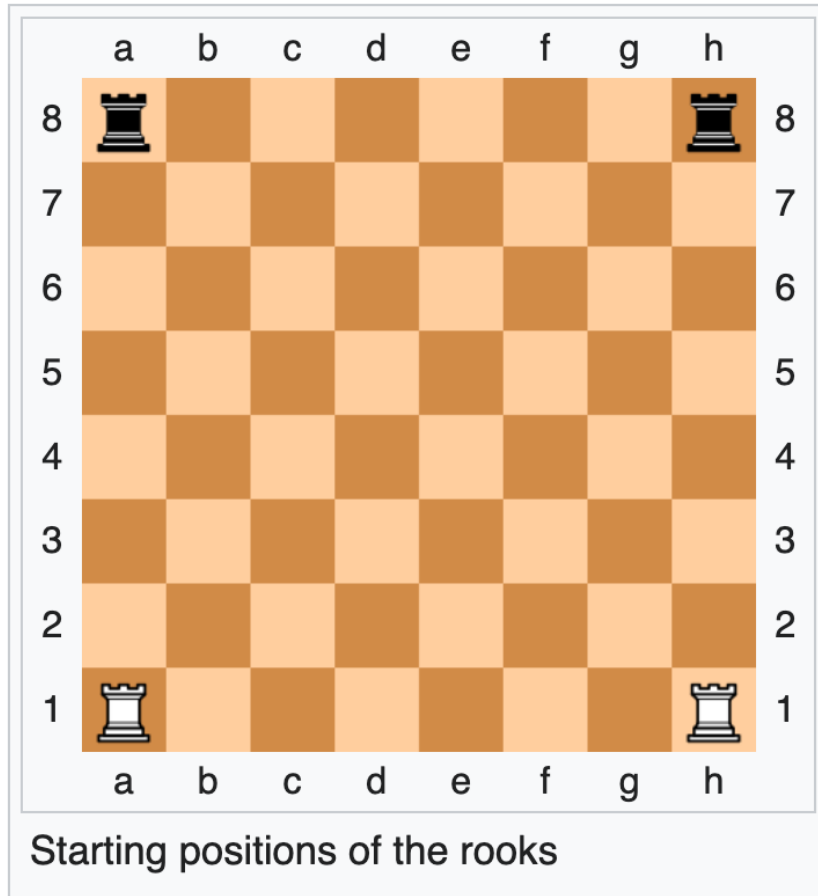
- A contiguous pair of spatial objects are those that share a common border.
- There are different ways that objects can “share a common border”.
- Rook contiguity requires that the pair of polygons in question share an edge.

P-06	P-07	P-08
P-03	P-04	P-05
P-00	P-01	P-02



Contiguity weights

- Rook contiguity requires that the pair of polygons in question share an edge.

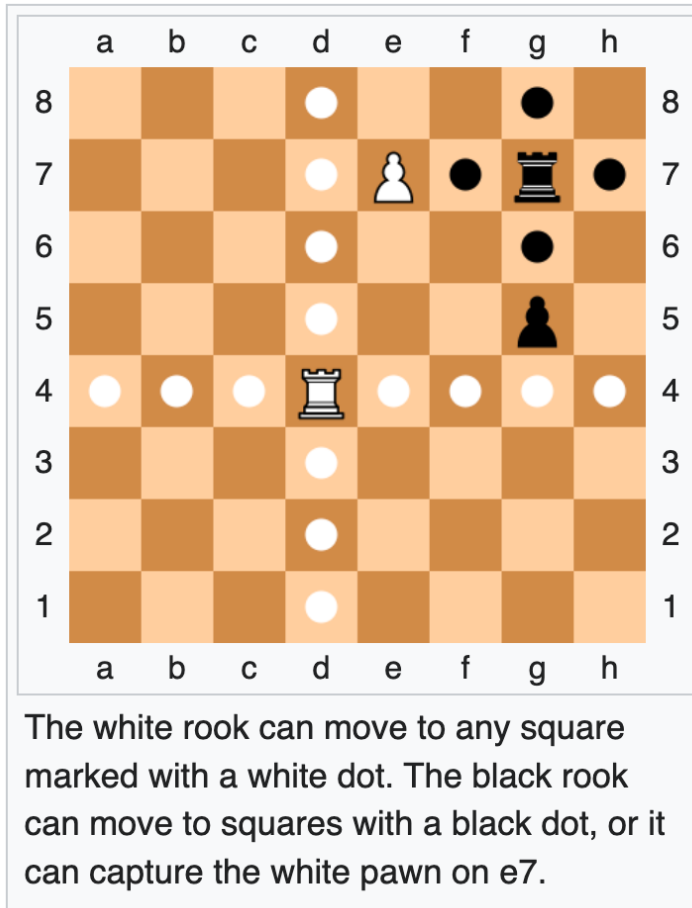


P-06	P-07	P-08
P-03	P-04	P-05
P-00	P-01	P-02



Contiguity weights

- Rook contiguity requires that the pair of polygons in question share an edge.

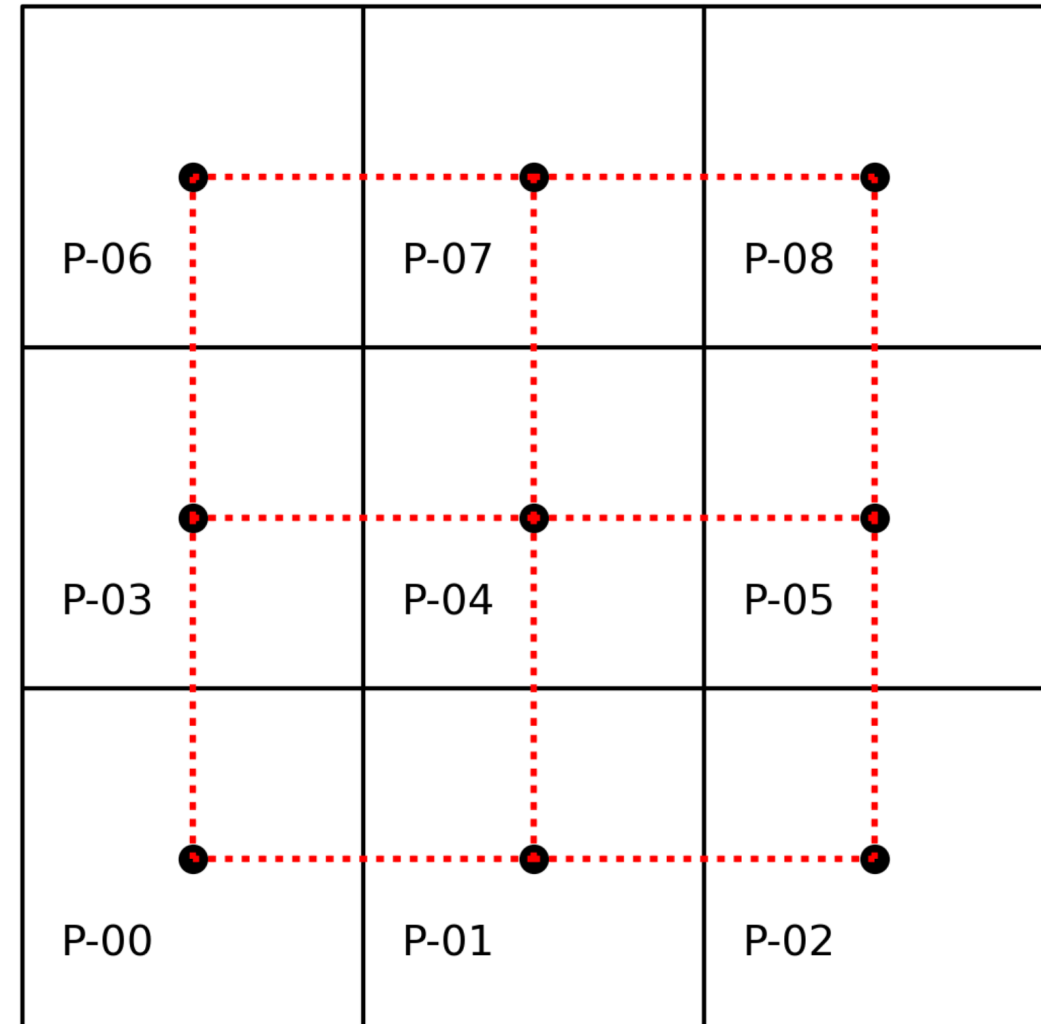
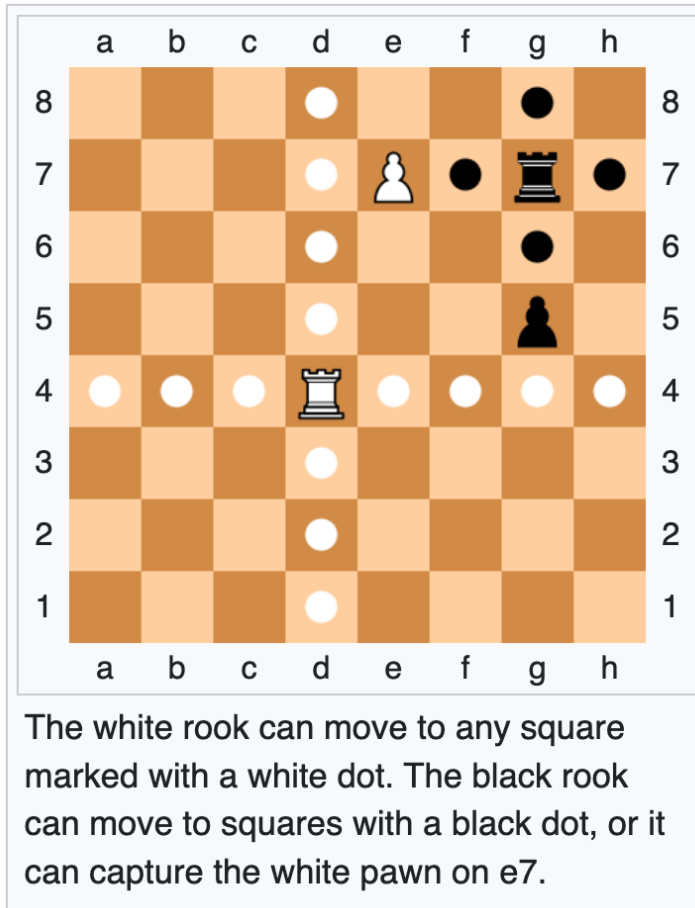


P-06	P-07	P-08
P-03	P-04	P-05
P-00	P-01	P-02



Contiguity weights

- Rook contiguity requires that the pair of polygons in question share an edge.



Contiguity weights

```
# Build a rook contiguity matrix from a regular 3x3  
# lattice stored in a geo-table  
wr = weights.contiguity.Rook.from_dataframe(gdf)
```

```
pandas.DataFrame(*wr.full()).astype(int)
```

	0	1	2	3	4	5	6	7	8
0	0	0	1	0	1	0	0	0	0
1	1	1	0	1	0	1	0	0	0
2	0	1	0	0	0	1	0	0	0
3	1	0	0	0	1	0	1	0	0
4	0	1	0	1	0	1	0	1	0
5	0	0	1	0	1	0	0	0	1
6	0	0	0	1	0	0	0	1	0
7	0	0	0	0	1	0	1	0	1
8	0	0	0	0	0	1	0	1	0

wr.nonzero

24

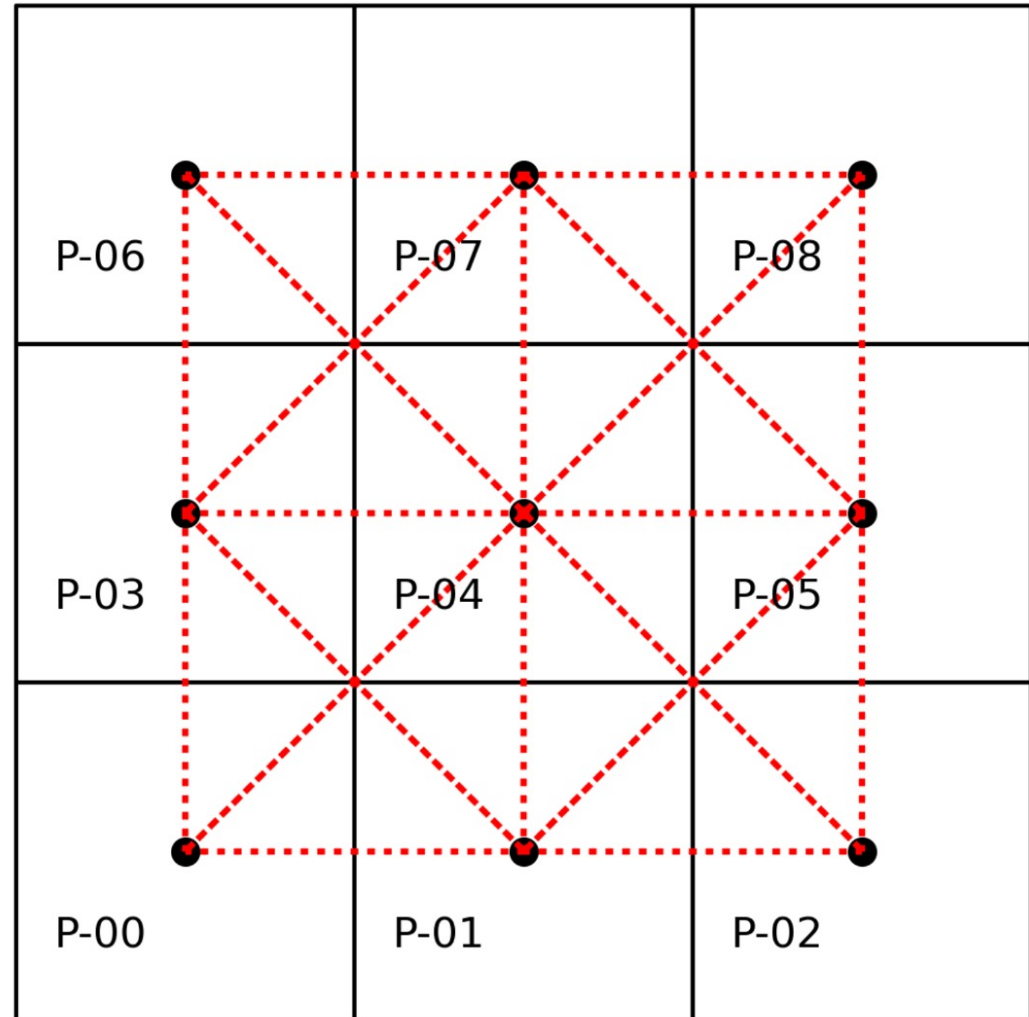
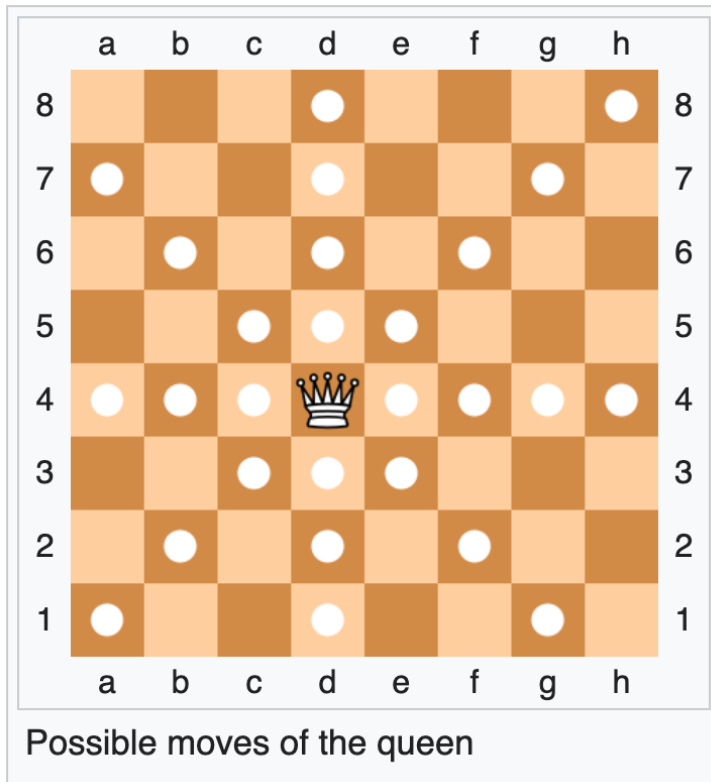
wr.neighbors

```
{0: [1, 3],  
 1: [0, 2, 4],  
 2: [1, 5],  
 3: [0, 4, 6],  
 4: [1, 3, 5, 7],  
 5: [8, 2, 4],  
 6: [3, 7],  
 7: [8, 4, 6],  
 8: [5, 7]}
```



Contiguity weights

- Queen contiguity requires a pair of polygons to only share one or more vertices.



Contiguity neighbors

```
# Build a queen contiguity matrix from a regular 3x3  
# lattice stored in a geo-table  
wq = weights.contiguity.Queen.from_dataframe(gdf)  
wq.neighbors
```

```
{0: [1, 3, 4],  
 1: [0, 2, 3, 4, 5],  
 2: [1, 4, 5],  
 3: [0, 1, 4, 6, 7],  
 4: [0, 1, 2, 3, 5, 6, 7, 8],  
 5: [1, 2, 4, 7, 8],  
 6: [3, 4, 7],  
 7: [3, 4, 5, 6, 8],  
 8: [4, 5, 7]}
```



Contiguity weights

- Similar to the `neighbors` attribute, the `weights` object is a Python dictionary that only stores the non-zero weights.

```
wq.weights
```

```
{0: [1.0, 1.0, 1.0],  
 1: [1.0, 1.0, 1.0, 1.0, 1.0],  
 2: [1.0, 1.0, 1.0],  
 3: [1.0, 1.0, 1.0, 1.0, 1.0],  
 4: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0],  
 5: [1.0, 1.0, 1.0, 1.0, 1.0],  
 6: [1.0, 1.0, 1.0],  
 7: [1.0, 1.0, 1.0, 1.0, 1.0],  
 8: [1.0, 1.0, 1.0]}
```



Cardinalities

- In addition to the `neighbor` and `weights` attributes, the `w` object has a large number of other attributes and methods that can be useful. The `cardinalities` attribute reports the number of neighbors for each observation:

```
wq.cardinalities
```

```
{0: 3, 1: 5, 2: 3, 3: 5, 4: 8, 5: 5, 6: 3, 7: 5, 8: 3}
```



```
wq.histogram
```

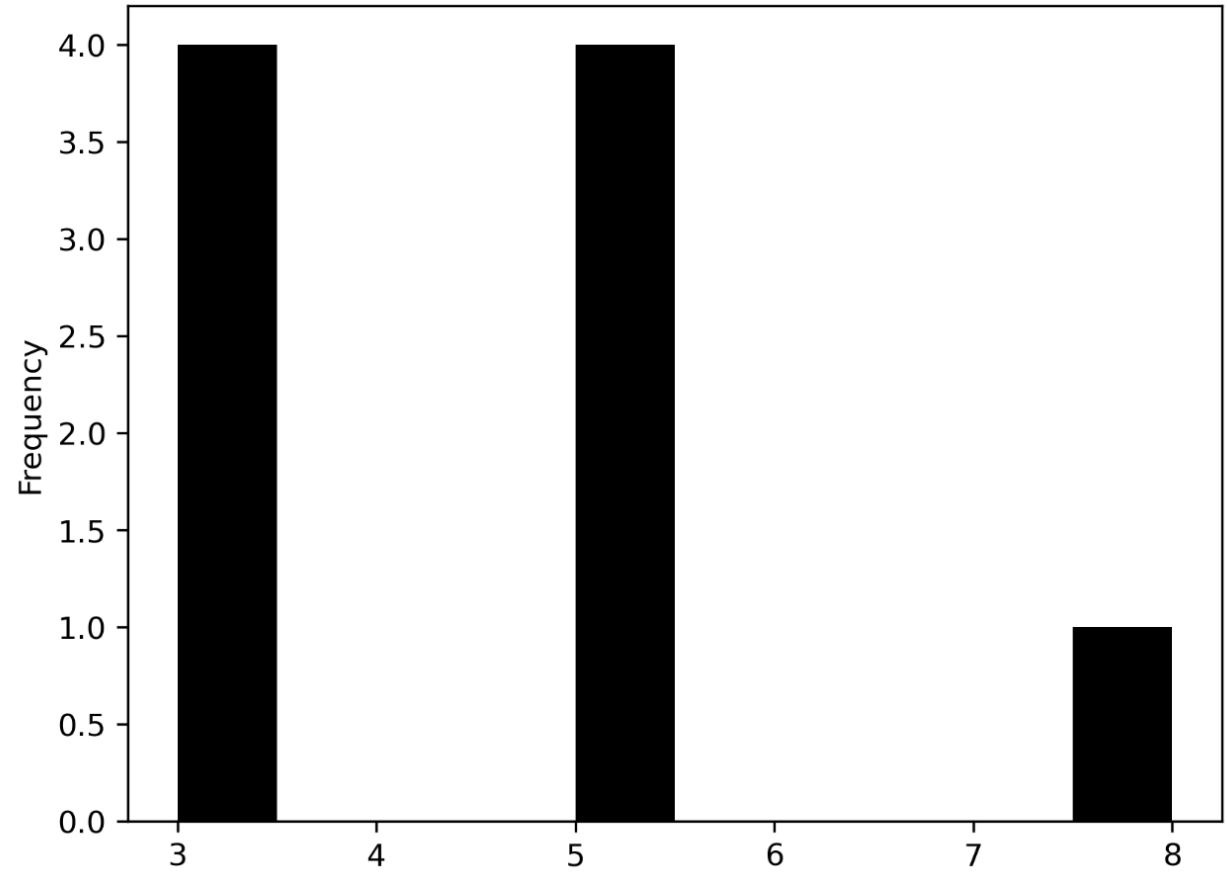
```
[(3, 4), (4, 0), (5, 4), (6, 0), (7, 0), (8, 1)]
```



Cardinalities

- We can obtain a quick visual representation by converting the cardinalities into a `pandas.Series` and creating a histogram.
- The distribution of the number of neighbours is asymmetric!

```
pandas.Series(wq.cardinalities).plot.hist(color="k");
```



Joins

- By convention, an ordered pair of contiguous observations constitutes a join represented by a non-zero weight in a W .
- The attribute $s0$ records the number of joins.

```
wq.s0
```

```
40.0
```

```
wq.pct_nonzero
```

```
49.382716049382715
```



Real-world example

```
san_diego_tracts = geopandas.read_file(  
    "../data/sandiego/sandiego_tracts.gpkg"  
)  
w_queen = weights.contiguity.Queen.from_dataframe(san_diego_tracts)
```

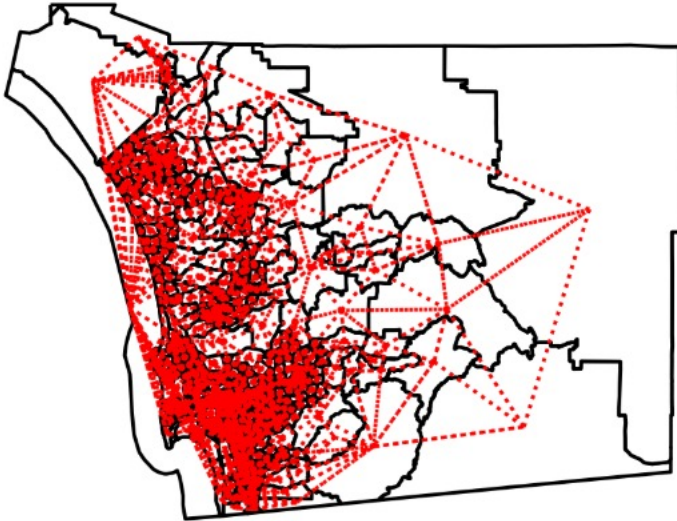
```
print(w_queen.n)  
print(w_queen.pct_nonzero)
```

```
628  
1.018296888311899
```



Real-world example

```
san_diego_tracts = geopandas.read_file(  
    "../data/sandiego/sandiego_tracts.gpkg"  
)  
w_queen = weights.contiguity.Queen.from_dataframe(san_diego_tracts)
```

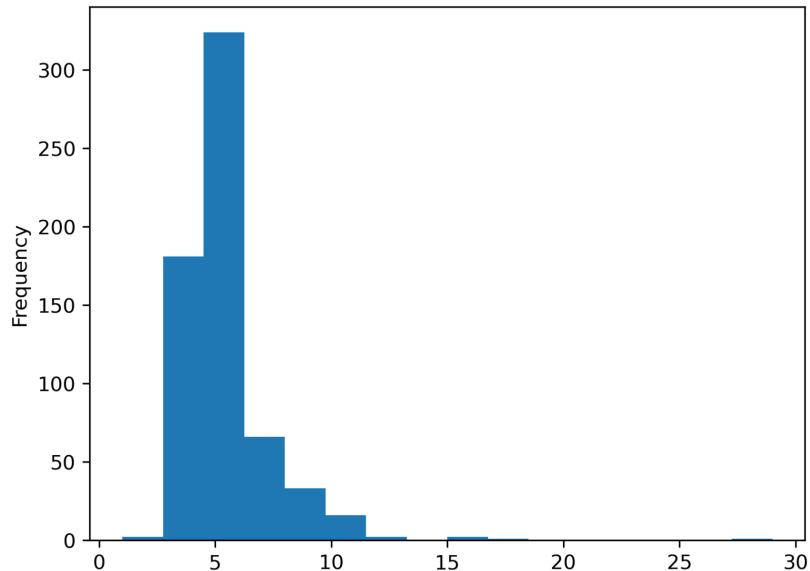


Real-world example

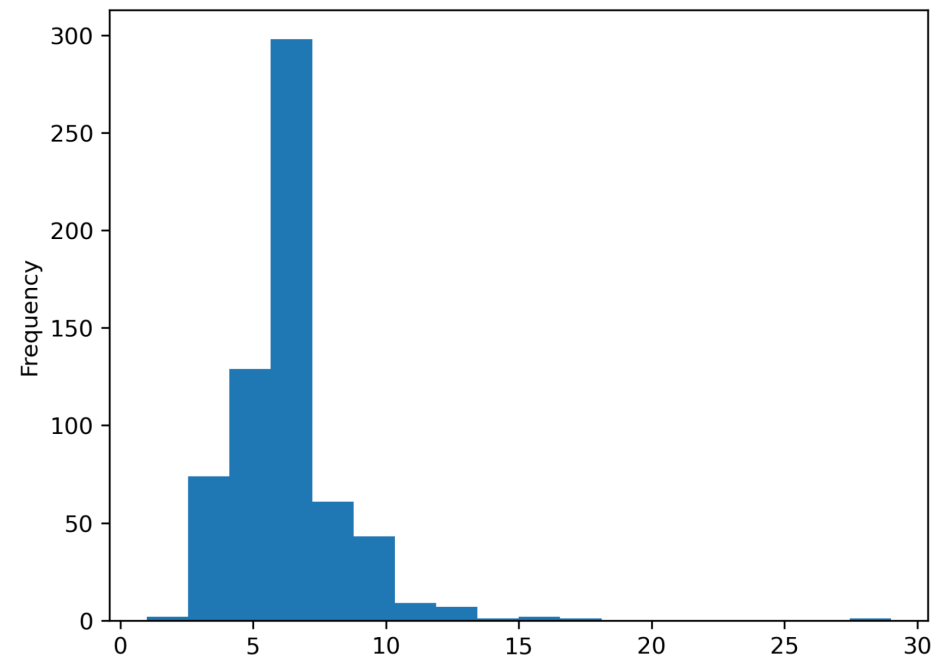
```
san_diego_tracts = geopandas.read_file(
    "../data/sandiego/sandiego_tracts.gpkg"
)
w_queen = weights.contiguity.Queen.from_dataframe(san_diego_tracts)
```

```
w_rook = weights.contiguity.Rook.from_dataframe(san_diego_tracts)
print(w_rook.pct_nonzero)
s = pandas.Series(w_rook.cardinalities)
s.plot.hist(bins=s.unique().shape[0]);
```

0.8722463385938578



```
s = pandas.Series(w_queen.cardinalities)
s.plot.hist(bins=s.unique().shape[0]);
```



Remarks

- This example shows how the notion of contiguity, although more straightforward in the case of a grid, can be naturally extended beyond the particular case of a regular lattice.
- The principle to keep in mind is that we consider contiguous (and hence call neighbors) observations which share part of their border coordinates.
- In the Queen case, a single point is enough to make the join. For Rook neighbors, we require a join to consist of one or more shared edges.
- This distinction is less relevant in the real world than it appears in the grid example above.
- In any case, there are some cases where this distinction can matter and it is useful to be familiar with the differences between the two approaches.



Spatial weights from surfaces

Most often, we will use spatial weights as a way to connect features stored in rows of a geographic table. A more niche application is spatial weights derived from surfaces.

Spatial weights from surfaces

- For the illustration, we will use a surface that contains population counts for the Sao Paulo region in Brazil:

```
sao_paulo = rioxarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.tif")
```

- From version 2.4 onwards, pysal added support to build spatial weights from xarray.DataArray objects.

```
w_sao_paulo = weights.contiguity.Queen.from_xarray(sao_paulo)
```



Distance based weights

In addition to contiguity, we can also define neighbor relations as a function of the distance separating spatial observations.

K-nearest neighbor weights

- The first type of distance based weights defines the neighbor set of a particular observation as containing its nearest k observations, where the user specifies the value of k .
- To illustrate this for the San Diego tracts, we take $k = 4$.

```
wk4 = weights.distance.KNN.from_dataframe(san_diego_tracts, k=4)
```

- Since we are dealing with polygons in this case, `pysal` uses inter-centroid distances to determine the k nearest observations to each polygon.



K-nearest neighbor weights

- The k-nearest neighbor weights displays no island problem, that is everyone has at least one neighbor:

```
wk4.islands
```

```
[]
```

```
wk4.histogram
```

```
[(4, 628)]
```



Kernel weights

- The k-nearest neighbor rule assigns binary values to the weights for neighboring observations.
- `pysal` also supports continuously valued weights to reflect Tobler's first law in a more direct way: *observations that are close to a unit have larger weights than more distant observations.*
- The essence of kernel weights is that the weight between observations and is based on their distance, but it is further modulated by a kernel function with certain properties.



Kernel weights

- The kernel weights are defined as a function $K(z)$ of the ratio between the distance d_{ij} from i to j , and the bandwidth h_i , with

$$z = d_{ij} / h_i$$

- This ensures that z is always less than 1. For distances greater than the bandwidth, $K(z) = 0$.



Kernel weights

- Usual kernel weights include:
 - Triangular: $K(z) = (1 - |z|)$ for $|z| < 1$. [default]
 - Uniform: $K(z) = 1/2$ for $|z| < 1$.
 - Quadratic: $K(z) = \left(\frac{3}{4}\right) (1 - z^2)$ for $|z| < 1$.
 - Quartic: $K(z) = \left(\frac{15}{16}\right) (1 - z^2)^2$ for $|z| < 1$.
 - Gaussian: $K(z) = (2\pi)^{-1/2} \exp\left(-\frac{z^2}{2}\right)$ for $|z| < 1$.



Kernel weights: Fixed bandwidth

- The default values for kernels are to use a triangular kernel with a bandwidth distance equal to the maximum $k_{nn} = 2$ distance for all observations.
- This is an example of fixed bandwidth where all observations use the same distance for the cut-off.

```
w_kernel = weights.distance.Kernel.from_dataframe(gdf)
```

```
w_kernel.function
```

```
'triangular'
```

```
# Show the first five values of bandwidths  
w_kernel.bandwidth[0:5]
```

```
array([[1.0000001],  
       [1.0000001],  
       [1.0000001],  
       [1.0000001],  
       [1.0000001]])
```



Kernel weights: Adaptive bandwidth

- A fixed bandwidth is not always the best choice.
- For example, in cases where the density of the observations varies over the study region, using the same threshold anywhere will result in regions with a high density of neighbors while others with observations very sparsely connected.
- In these situations, an **adaptive bandwidth** – one which varies by observation and its characteristics – can be preferred.



Kernel weights: Adaptive bandwidth

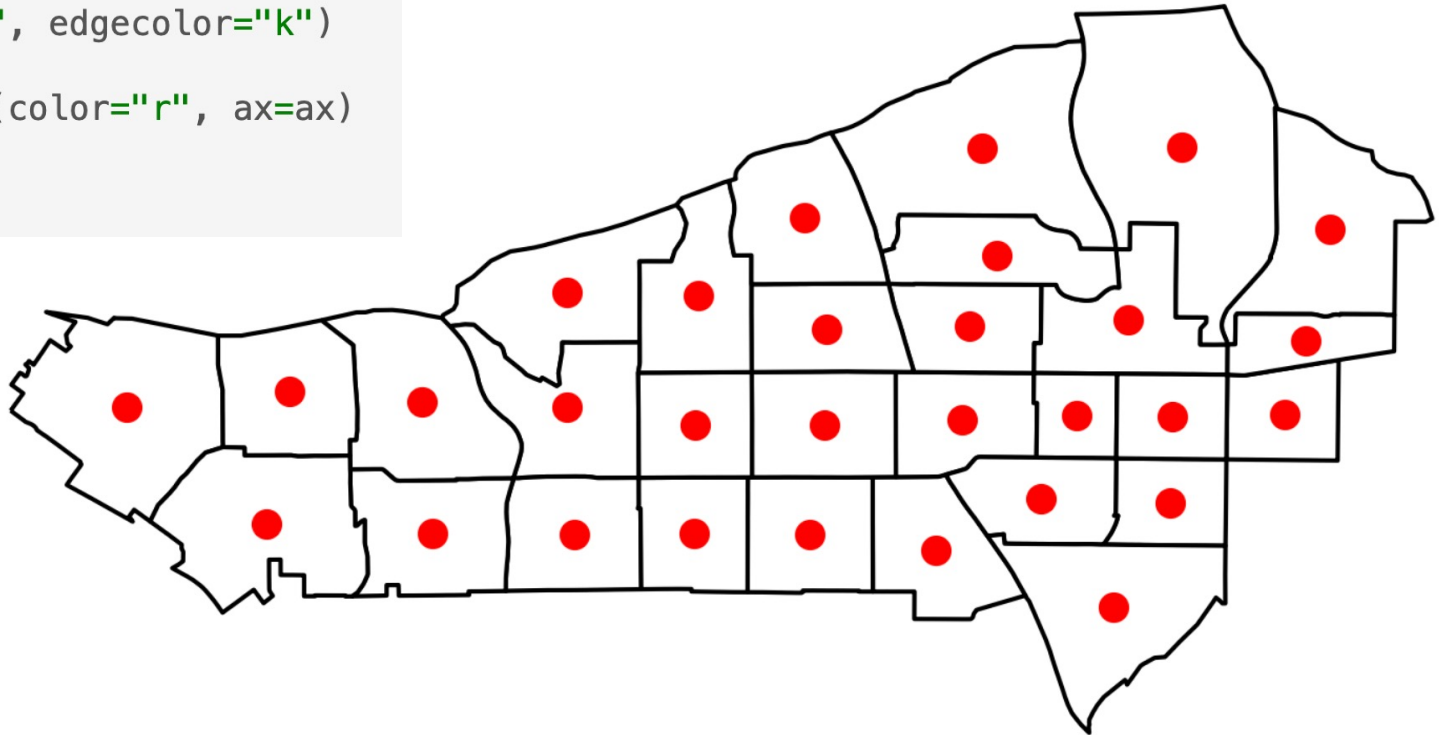
- Adaptive bandwidths can be picked again using a K-nearest neighbor rule.
 - A bandwidth for each observation is chosen such that, once the k-nearest observation is considered, all the remaining observations have zero weight.



Kernel weights: Adaptive bandwidth

- To illustrate it, we will use a subset of tracts in our San Diego dataset. First, visualizing the centroids, we can see that they are not exactly regularly-spaced, although others do nearly fall into a regular spacing:

```
# Create subset of tracts
sub_30 = san_diego_tracts.query("sub_30 == True")
# Plot polygons
ax = sub_30.plot(facecolor="w", edgecolor="k")
# Create and plot centroids
sub_30.head(30).centroid.plot(color="r", ax=ax)
# Remove axis
ax.set_axis_off();
```



Kernel weights: Adaptive bandwidth

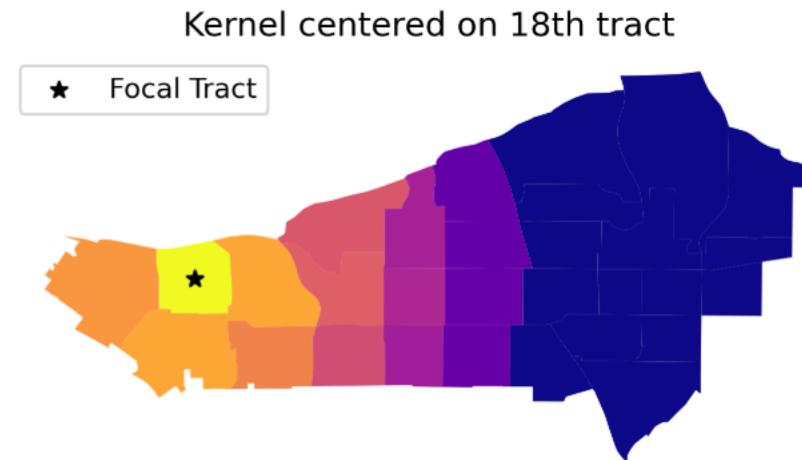
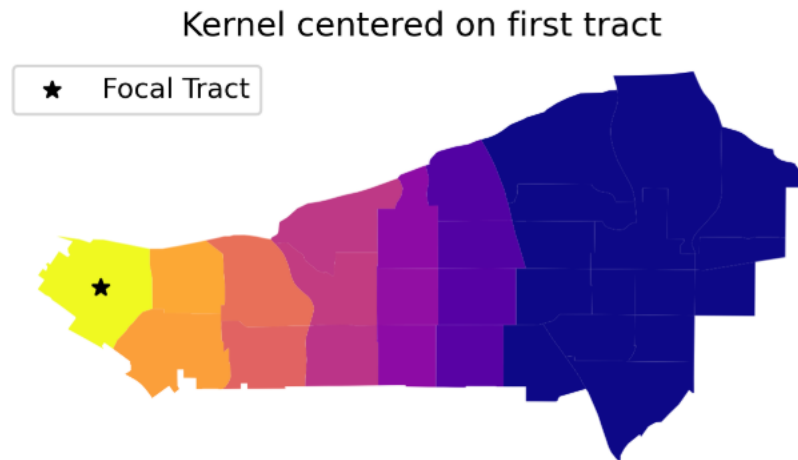
```
# Build weights with adaptive bandwidth
w_adaptive = weights.distance.Kernel.from_dataframe(
    sub_30, fixed=False, k=15
)
# Print first five bandwidth values
w_adaptive.bandwidth[:5]
```

```
array( [[7065.74020822],
        [3577.22591841],
        [2989.74807871],
        [2891.46196945],
        [3965.08354232]] )
```



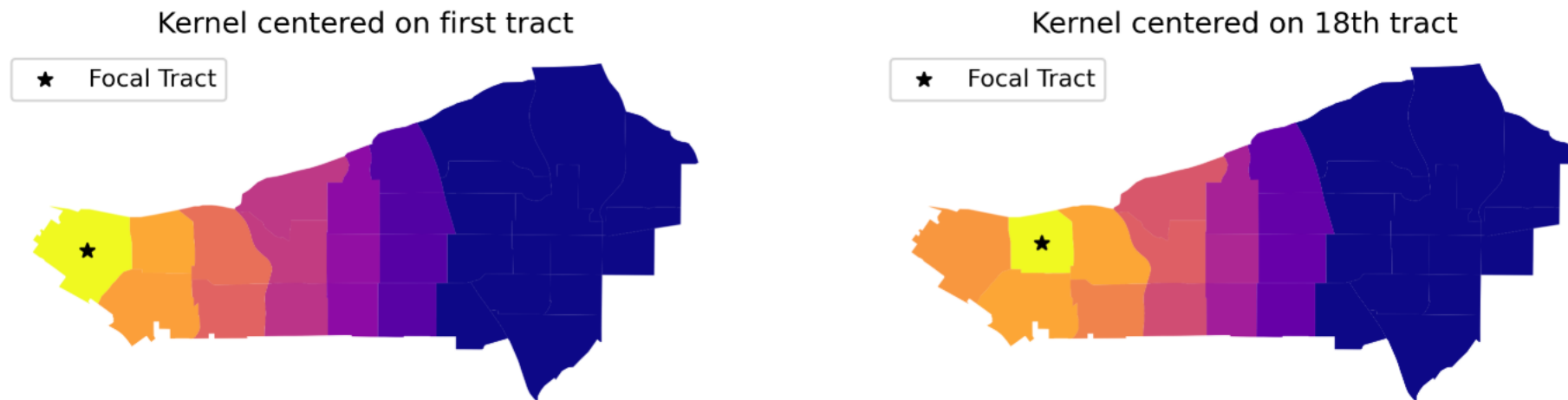
Kernel weights: Adaptive bandwidth

- And, we can visualize what these kernels look like on the map, too, by focusing on an individual unit and showing how the distance decay attenuates the weight by grabbing the corresponding row of the full kernel matrix:



Kernel weights: Adaptive bandwidth

- What the kernel looks like can be strongly affected by the structure of spatial proximity, so any part of the map can look quite different from any other part of the map. By imposing a clear distance decay over several of the neighbors of each observation, kernel weights incorporate Tobler's law explicitly.



Often, this comes at the cost of increased memory requirements, as every single pair of observations within the bandwidth distance is considered:



Kernel weights: Adaptive bandwidth

- In many instances, this may be at odds with the nature of the spatial interactions at hand, which operate over a more limited range of distance. In these cases, expanding the neighborhood set beyond might lead us to consider interactions which do not take place, or are inconsequential.
- Thus, for both substantive and computational reasons, it might make sense to further limit the range, keeping impacts hyper-local.



Distance bands and hybrid Weights

- In some contexts, it makes sense to draw a circle around each observation and consider as neighbors every other observation that falls within the circle.

```
w_bdb = weights.distance.DistanceBand.from_dataframe(  
    gdf, 1.5, binary=True  
)
```

- This creates a binary distance weights where every other observation within a distance of 1.5 is considered neighbor.



Distance bands and hybrid Weights

- Distance band weights can also be continuously weighted.
- For example, let us calculate the DistanceBand weights that use inverse distance weights up to a certain threshold and then truncate the weights to zero for everyone else.

```
w_hy = weights.distance.DistanceBand.from_dataframe(  
    gdf, 1.5, binary=False  
)
```

- We apply a threshold of 1.5 for this illustration. `pysal` truncates continuous weights at this distance.



Great circle distances

- It may in some cases be very important that the calculation of distances between objects takes the geographic reference system into account.
- This can be done before computing the spatial weights object, by transforming the coordinates of data points into a projected reference system.



Block weights

A final type of spatial weight we examine here is block weights. In this case, it is membership in a geographic group that defines the neighbor relationships.

Block weights

- Block weights connect every observation in a dataset that belongs to the same category in a provided list.
- In essence, a block weight structure groups individual observations and considers all members of the group as “near” one another.
 - This means that they then have a value of one for every pair of observations in the same group. Contrariwise, all members *not* in that group are considered disconnected from any observation within the group, and given a value of zero.



Block weights

- Every tract has a unique ID (GEOID) and a county ID, shared by all tracts in the same county.

```
# NOTE: since this is a large dataset, it might take a while to process
w_bl = weights.util.block_weights(
    san_diego_tracts["county"].values,
    ids=san_diego_tracts["GEOID"].values,
)
```

- We can use block weights as an intermediate step in more involved analyses of “hybrid” spatial relationships.



Block weights

```
san_diego_tracts[["GEOID", "state", "county", "tract"]].head()
```

	GEOID	state	county	tract
0	06073018300	06	073	018300
1	06073018601	06	073	018601
2	06073017601	06	073	017601
3	06073019301	06	073	019301
4	06073018700	06	073	018700



Set operations on weights

So far, we have seen different principles that guide how to build spatial weights matrices. In this section, we explore how we can create new matrices by combining different existing ones.

Editing/connecting disconnected observations: San Diego tracts

- Imagine one of our tracts was an island and did not have any neighbors in the contiguity case. This can create issues in the spatial analytics that build on spatial weights, so it is good practice to amend the matrix before using it.
- The first approach we adopt is to find the nearest neighbor for the island observation and then add this pair of neighbors to extend the neighbor pairs from the original contiguity weight to obtain a fully connected set of weights.



Editing/connecting disconnected observations: San Diego tracts

- We will assume, for the sake of the example, that the disconnected observation (or the one are interested to edit) was number 103.

```
disconnected_tract = san_diego_tracts.iloc[[103]]
```

- We will connect the observation to its nearest neighbor. To do this, we can construct the KNN graph as we did above, but set $k=1$, so observations are only assigned to their nearest neighbor:

```
wk1 = weights.distance.KNN.from_dataframe(san_diego_tracts, k=1)
```

```
wk1.neighbors[103]
```

```
[102]
```



Editing/connecting disconnected observations: San Diego tracts

- To connect it in our initial matrix, we need to create a copy of the neighbors dictionary and update the entry for 103, including 102 as a neighbor. We copy the neighbors:

```
neighbors = w_queen.neighbors.copy()
```

```
neighbors[103].append(102)  
neighbors[102].append(103)  
w_new = weights.W(neighbors)  
w_new[103]
```

```
{480: 1.0, 160: 1.0, 324: 1.0, 102: 1.0, 107: 1.0, 173: 1.0}
```



Using the union of matrices

- A more elegant approach to the island problem makes use of `pysal`'s support for set theoretic operations on `pysal` weights. For example, we can construct the union of two weighting schemes, connecting any pair of observations if they are connected in either the Rook or if they are nearest neighbors:

```
w_fixed_sets = weights.set_operations.w_union(w_rook, wk1)
```



Visualizing weight set operations

- To further build the intuition behind different criteria, in this section we illustrate these concepts using the 32 states of Mexico.
- Queen and KNN graphs are relatively similar but, as one would expect, the KNN is sparser than Queen in areas with high density of irregular polygons

